

Informe #1

Definición del problema



Integrantes:
Owuen Yagual Panimboza
Stiven Rivera Palma

29 de Agosto, 2026

Contents

1	Introduction	2
1.1	Descripción del problema	2
1.2	Justificación del problema	3
1.3	Objetivos	3
1.3.1	Objetivo principal	3
1.3.2	Objetivos específicos	3
1.4	Módulos del proyecto	3
1.4.1	Módulo 1 – Captura y Preprocesamiento de Imagen	4
1.4.2	Módulo 2 – Detección de Pose con MediaPipe	4
1.4.3	Módulo 3 – Cálculo de Ángulos Articulares	5
1.4.4	Módulo 4 – Simulación del Brazo Robótico Virtual	6
1.4.5	Módulo 5 – Visualización y Métricas en Tiempo Real	7
1.5	Soluciones existentes	7
	References	9

Chapter 1

Introduction

1.1 Descripción del problema

La captura de movimiento markerless (sin marcadores físicos) representa una línea activa de investigación en visión artificial con aplicaciones en robótica, rehabilitación médica, interfaces humano-máquina e interacción natural. [1]

Los enfoques tradicionales basados en sensores físicos, trajes de captura óptica o cámaras de profundidad resultan costosos e intrusivos. El presente proyecto propone un sistema basado exclusivamente en una cámara convencional que, procesando el video en tiempo real, detecte la pose del brazo humano, calcule los ángulos articulares de hombro, codo y muñeca, y transfiera esos valores a la simulación visual de un brazo robótico virtual de tres grados de libertad (GDL). [2]

La solución adopta MediaPipe Pose de Google, un framework de código abierto que proporciona detección de 33 landmarks corporales en tiempo real sin necesidad de GPU, combinado con OpenCV para la adquisición y el procesamiento de imagen. [3] [4]

Diagrama de Bloques General del Sistema

La figura 1. representa el flujo de datos global del proyecto, descompuesto en sus cinco módulos funcionales principales.

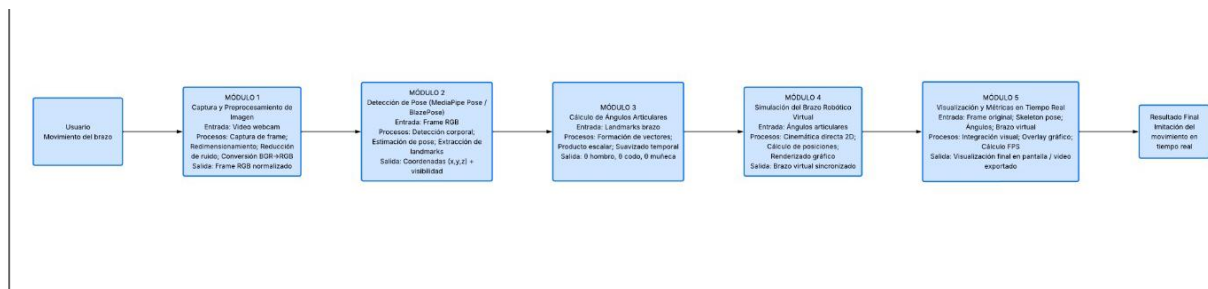


Figure 1. – Diagrama de bloques del sistema de imitación de movimientos con MediaPipe Pose.

Cada bloque representa un módulo funcional independiente con entradas y salidas definidas. El video de la cámara entra al sistema por M1, los landmarks son extraídos en M2, los ángulos se calculan en M3, el brazo robótico es simulado en M4 y todo se integra visualmente en M5. [3] [6]

1.2 Justificación del problema

MediaPipe fue presentado originalmente como un framework modular para construir pipelines de percepción reproducibles entre plataformas. Su componente BlazePose amplió esta capacidad al dominio de la estimación de pose corporal en tiempo real, logrando latencias menores a 30 ms en CPU mediante un pipeline de dos etapas: detector de persona (BlazePose Detector) y estimador de pose (BlazePose Estimator) que extrae 33 puntos clave. [3] [4]

Trabajos previos como los de Pauzi et al. (2021) demostraron la efectividad de BlazePose para estimar movimientos corporales con una diferencia de precisión dentro del 10% respecto a sistemas IMU de referencia. En el dominio de la teleoperación robótica, Altayeb (2023) implementó un sistema de replicación de gestos de mano en un brazo robótico usando MediaPipe Hands, validando la viabilidad de este enfoque para control de actuadores virtuales. [1] [5]

La justificación económica radica en el bajo costo de implementación: el sistema requiere únicamente una webcam USB estándar (< \$20 USD), sin sensores de profundidad ni hardware especializado. El stack de software es completamente open-source (Python, OpenCV, MediaPipe, NumPy).

1.3 Objetivos

1.3.1 Objetivo principal

Desarrollar un sistema de visión artificial que, a partir de video capturado en tiempo real por una cámara convencional, detecte los movimientos del brazo humano utilizando MediaPipe Pose [3] [4], calcule los ángulos de las articulaciones hombro, codo y muñeca, y los transfiera a la simulación visual sincronizada de un brazo robótico virtual de 3 GDL.

1.3.2 Objetivos específicos

1. Implementar un módulo de captura y preprocesamiento de video en tiempo real usando OpenCV [6] con soporte para normalización de resolución (640×480) y conversión de espacio de color BGR→RGB.
2. Integrar el modelo MediaPipe Pose [3] para la detección y extracción de los 33 landmarks corporales, con énfasis en los puntos clave del brazo (hombros: landmarks 11,12; codos: 13,14; muñecas: 15,16).
3. Desarrollar un módulo de cálculo de ángulos articulares mediante álgebra vectorial (producto escalar y norma de vectores) para los tres GDL del brazo: hombro (elevación), codo (flexión) y muñeca (flexión), con filtrado de media móvil de 5 frames.
4. Implementar la simulación 2D del brazo robótico virtual mediante cinemática directa, renderizando en tiempo real el modelo sobre el frame de video usando primitivas OpenCV (cv2.line, cv2.circle) [6].
5. Construir un módulo de visualización integrada que muestre el skeleton de MediaPipe [4], los ángulos articulares calculados, el brazo robótico simulado y métricas de rendimiento (FPS).
6. Evaluar el desempeño del sistema con métricas de latencia (FPS objetivo: ≥ 20 FPS), precisión angular y estabilidad del sistema ante variaciones de iluminación, usando como referencia el benchmark del dataset COCO Keypoints [7].

1.4 Módulos del proyecto

El sistema se descompone en cinco módulos funcionales interconectados. La arquitectura modular facilita el desarrollo y prueba independiente de cada componente en el marco SCRUM adoptado. Las tecnologías principales son OpenCV [6] para procesamiento de imagen y MediaPipe Pose [3, 7] para estimación de pose.

1.4.1 Módulo 1 – Captura y Preprocesamiento de Imagen

Objetivo

Adquirir el flujo de video en tiempo real desde la cámara y preparar cada frame para su procesamiento posterior mediante normalización de resolución, conversión de espacio de color y reducción de ruido.

	Descripción	Formato / Especificación
Entrada	Flujo de video desde webcam USB o cámara integrada	Resolución variable (640×480 – 1920×1080), 30 FPS
Salida	Frame de imagen normalizado en espacio RGB	NumPy array RGB, resolución fija 640×480, píxeles en [0, 255]

Table 1. – Entradas y salidas del Módulo 1.

Entradas y Salidas

Herramientas Tecnológicas

- **Software:** Python 3.10+ como lenguaje de implementación.
- **Librería principal:** OpenCV (cv2 v4.8+) [6] — captura de video (cv2.VideoCapture), redimensionado (cv2.resize) y conversión de espacio de color (cv2.cvtColor).
- **Librería de soporte:** NumPy (v1.24+) para manejo y operaciones sobre arrays de imagen.

Técnicas de Visión Computacional

- **Realzado y restauración:** aplicación opcional de filtro gaussiano (cv2.GaussianBlur) para atenuación de ruido de alta frecuencia antes de la inferencia de MediaPipe [8].
- Redimensionado con interpolación bicúbica (cv2.INTER_CUBIC) para preservar calidad de bordes al escalar el frame.
- Conversión de espacio de color BGR→RGB requerida por el modelo BlazePose de MediaPipe.

1.4.2 Módulo 2 – Detección de Pose con MediaPipe

Objetivo

Detectar los 33 landmarks del cuerpo humano en cada frame de video utilizando el modelo MediaPipe Pose (BlazePose), con énfasis en los puntos clave del brazo: hombros (landmarks 11, 12), codos (13, 14) y muñecas (15, 16) [3].

Entradas y Salidas

Herramientas Tecnológicas

- **Modelo de IA preentrenado:** MediaPipe Pose (BlazePose) v0.10+ [3, 4] — modelo de estimación de pose de 33 landmarks entrenado sobre el dataset COCO Keypoints. No requiere GPU.

	Descripción	Formato / Especificación
Entrada	Frame RGB preprocesado (salida de M1)	NumPy array RGB 640×480
Salida	33 landmarks corporales con visibilidad	NormalizedLandmarkList: coord. (x, y, z) $\in [0, 1]$ + visibilidad $\in [0, 1]$ por punto

Table 2. – Entradas y salidas del Módulo 2.

- **Framework:** MediaPipe (Google) [4] — framework de construcción de pipelines de percepción con soporte cross-platform.
- **Librería de soporte:** NumPy para extracción y transformación de las coordenadas de landmarks al espacio de píxeles.

Técnicas de Visión Computacional

- **Pipeline BlazePose de dos etapas:**
 1. Detector de persona que delimita el bounding box de la región de interés.
 2. Estimador de pose que infiere los 33 landmarks con alta precisión dentro de esa región.
- **Modo de video** (`static_image_mode=False`): el modelo aprovecha la coherencia temporal entre frames consecutivos para acelerar la inferencia y estabilizar los landmarks.
- **Extracción de landmarks de interés:** se seleccionan los índices 11–16 (hombros, codos, muñecas) de los 33 puntos detectados para alimentar el módulo de cálculo angular.

1.4.3 Módulo 3 – Cálculo de Ángulos Articulares

Objetivo

Calcular los ángulos de flexión/extensión en las articulaciones hombro, codo y muñeca a partir de las coordenadas 2D de los landmarks, usando la definición vectorial del ángulo entre dos segmentos articulares.

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Coordenadas 2D de hombro, codo y muñeca (salida de M2)	Pares (x, y) en píxeles para cada articulación (landmarks 11–16)
Salida	Tripleta de ángulos articulares suavizados	$\theta_{\text{hombro}}, \theta_{\text{codo}}, \theta_{\text{muñeca}}$ en grados $[0, 180]$ (float64)

Table 3. – Entradas y salidas del Módulo 3.

Herramientas Tecnológicas

- **Librería principal:** NumPy (v1.24+) para cálculo del producto escalar (`np.dot`), norma de vectores (`np.linalg.norm`) y función arccos (`np.arccos`) necesarios en la fórmula angular.
- **Librería de soporte:** Matplotlib (v3.7+) para graficar la evolución temporal de los ángulos articulares durante las pruebas y análisis de resultados.

Técnicas de Visión Computacional y Geometría

- **Cálculo de ángulos mediante producto escalar:** [8]

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{|\mathbf{v}_1| |\mathbf{v}_2|}\right) \quad (1.1)$$

donde $\mathbf{v}_1$ y $\mathbf{v}_2$ son los vectores que forman cada par de segmentos en la articulación.

- **Filtrado temporal con media móvil** de ventana $w = 5$ frames para suavizar fluctuaciones por ruido de detección de BlazePose y evitar movimientos bruscos en la simulación del brazo robótico.
- **Cálculo de vectores entre articulaciones:** diferencia de coordenadas entre el landmark proximal y el distal para construir cada vector de segmento corporal.

1.4.4 Módulo 4 – Simulación del Brazo Robótico Virtual

Objetivo

Representar y animar en pantalla un brazo robótico de 3 GDL cuyas posiciones articulares se corresponden directamente con los ángulos calculados por M3, logrando una imitación visual sincronizada en tiempo real [5].

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Ángulos articulares suavizados (salida de M3)	$\theta_{\text{hombro}}, \theta_{\text{codo}}, \theta_{\text{muñeca}}$ en grados (float64)
Salida	Frame con overlay del brazo robótico virtual	NumPy array BGR con eslabones y articulaciones renderizados en tiempo real

Table 4. – Entradas y salidas del Módulo 4.

Herramientas Tecnológicas

- **Librería principal:** OpenCV (cv2) [6] para renderizado de primitivas gráficas: cv2.line (eslabones), cv2.circle (articulaciones), cv2.putText (ángulos).
- **Librería de soporte:** NumPy para el cálculo de coordenadas cartesianas de cada articulación mediante cinemática directa 2D.

Técnicas de Visión Computacional y Robótica

- **Cinemática directa 2D:** a partir de los ángulos articulares y las longitudes de eslabón predefinidas ($L_{\text{hombro}}, L_{\text{antebrazo}}, L_{\text{mano}}$), se calculan las posiciones cartesianas (x, y) de cada junta del brazo virtual mediante relaciones trigonométricas acumulativas:

$$\begin{aligned} x_1 &= x_0 + L_{\text{hombro}} \cos(\theta_{\text{hombro}}) \\ y_1 &= y_0 - L_{\text{hombro}} \sin(\theta_{\text{hombro}}) \\ x_2 &= x_1 + L_{\text{antebrazo}} \cos(\theta_{\text{hombro}} + \theta_{\text{codo}}) \\ y_2 &= y_1 - L_{\text{antebrazo}} \sin(\theta_{\text{hombro}} + \theta_{\text{codo}}) \end{aligned} \quad (1.2)$$

- **Renderizado de overlay:** el brazo robótico se dibuja sobre el frame de video como una capa superpuesta, permitiendo la comparación visual directa entre el movimiento humano y la imitación del brazo virtual.

1.4.5 Módulo 5 – Visualización y Métricas en Tiempo Real

Objetivo

Integrar y mostrar en una ventana unificada el frame original anotado con el skeleton de MediaPipe, los ángulos calculados, el brazo robótico virtual y las métricas de rendimiento del sistema (FPS).

Entradas y Salidas

	Descripción	Formato / Especificación
Entrada	Frame original + landmarks + ángulos + overlay del brazo robótico	Salidas de M1, M2, M3 y M4 en cada iteración del bucle principal
Salida	Frame anotado final visualizado en tiempo real	Ventana OpenCV (cv2.imshow) con overlay completo; exportación opcional a MP4 (cv2.VideoWriter)

Table 5. – Entradas y salidas del Módulo 5.

Herramientas Tecnológicas

- **Librería principal:** OpenCV (cv2) [6] — visualización en ventana interactiva (cv2.imshow, cv2.waitKey) y exportación de video (cv2.VideoWriter).
- **MediaPipe Drawing Utils** [4] — dibujo automático del skeleton de 33 landmarks y conexiones corporales sobre el frame.
- Matplotlib (v3.7+) — generación de gráficas de evolución temporal de ángulos articulares y métricas de FPS para análisis de resultados.
- scikit-learn (v1.3+) — cálculo de métricas estadísticas de evaluación: error medio angular (MAE), correlación de Pearson y análisis de estabilidad del sistema.

Técnicas de Visión Computacional

- **Cálculo de FPS en tiempo real:** medición del tiempo de procesamiento por frame con timestamps de Python (time.time()) para monitoreo continuo del rendimiento del sistema.
- **Visualización compuesta:** superposición de múltiples capas (frame original, skeleton MediaPipe, overlay robótico, texto de métricas) mediante operaciones de blending de OpenCV.

1.5 Soluciones existentes

Hoy en día, replicar los movimientos del cuerpo humano mediante visión por computadora es un campo clave para disciplinas como la robótica, la interacción persona-ordenador, la medicina de rehabilitación y la automatización. En la literatura actual encontramos diferentes estrategias para resolver esto, pero casi todas comparten el mismo fin: capturar lo que hace un usuario, calcular dónde están sus articulaciones en el espacio y trasladar esa información a un robot o a un entorno digital.

Una de las líneas de investigación con más fuerza es la teleoperación por imitación. Aquí el reto principal no es solo registrar el movimiento de los brazos o piernas del usuario, sino ver cómo adaptar esos datos a la estructura de un robot, cuyas articulaciones y límites físicos suelen ser muy diferentes a los de un humano.

Por ejemplo, Zhang y su equipo diseñaron un sistema para conectar los movimientos de un brazo humano con un robot industrial. Lo interesante de su propuesta es que crearon un modelo cinemático capaz de traducir las trayectorias en tiempo real; el robot lograba replicar la orientación del brazo incluso teniendo una anatomía completamente distinta [9]. Esto demostró que la compatibilidad física no es una barrera insalvable. En una línea parecida, Xuan y Tang apostaron por la simplicidad del hardware al usar una sola cámara web común (óptica monocular) para controlar un brazo robótico [10]. Su trabajo dejó claro que no hacen falta sensores pegados al cuerpo ni trajes especiales si se cuenta con un buen procesamiento visual.

Por otro lado, cuando se trabaja con robots humanoides, la complejidad aumenta. En otro estudio, Han et al. se enfocaron en este problema desarrollando un método para rastrear específicamente el codo y la muñeca del operario. Su sistema ajusta esas coordenadas sobre la marcha para que el robot aprenda tareas de manipulación sin perder el equilibrio ni la fluidez [11]. Más recientemente, un estudio de la Universidad de los Andes (2026) propuso usar cámaras RGB-D (que miden profundidad) junto con algoritmos de estimación de pose. Su investigación confirmó que la visión artificial actual es lo suficientemente madura como para interactuar con gemelos virtuales y robots físicos con una precisión bastante aceptable, dejando atrás los incómodos sensores portables [12].

Al revisar todas estas soluciones, salta a la vista que casi todos los autores siguen una misma receta: capturan el video, detectan los puntos clave del cuerpo, analizan la geometría del movimiento y lo envían al dispositivo. Sin embargo, hay un problema: casi toda la investigación actual se empeña en usar robots físicos caros o equipos muy específicos. Es justo ahí donde este proyecto quiere marcar la diferencia. En lugar de lidiar con hardware costoso, proponemos crear una simulación con un brazo robótico virtual. De esta forma, reducimos los costos y la complejidad técnica a cero, pero manteniendo el foco en lo importante: experimentar y validar algoritmos de visión artificial y procesamiento de imágenes en tiempo real.

References

- [1] A. S. B. Pauzi, F. B. M. Nazri, S. Sani, A. M. Bataineh, M. N. Hisyam, M. H. Jaafar, M. N. Ab Wahab, and A. S. A. Mohamed, "Movement estimation using mediapipe blazepose," in *Advances in Visual Informatics. IVIC 2021*, vol. 13051 of *Lecture Notes in Computer Science*, pp. 590–600, Springer, Cham, 2021.
- [2] B. X. Vintimilla, "Propuestas de proyectos – curso dip 2026 pao-i." Diapositivas de presentación, 2026. Escuela Superior Politécnica del Litoral (ESPOL), Facultad de Ingeniería en Electricidad y Computación (FIEC).
- [3] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "Blazepose: On-device real-time body pose tracking," *arXiv*, 2020.
- [4] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "Mediapipe: A framework for building perception pipelines," *arXiv*, 2019.
- [5] M. Altayeb, "Hand gestures replicating robot arm based on mediapipe," *Indonesian Journal of Electrical Engineering and Informatics (IJEETI)*, vol. 11, pp. 727–737, 09 2023.
- [6] G. Bradski, "The opencv library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Computer Vision – ECCV 2014* (D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, eds.), (Cham), pp. 740–755, Springer International Publishing, 2014.
- [8] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Pearson, 4th ed., 2018.
- [9] Z. Wang, R. Liang, Z. Chen, and B. Liang, "Fast and intuitive kinematics mapping for human-robot motion imitating: A virtual-joint-based approach," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 10011–10018, 2020.
- [10] C. Xuan, Z. Tang, and J. Xu, "Vision-based robotic arm imitation by human gesture," *arXiv preprint arXiv:1703.04906*, 2017.
- [11] K. Han, S. Li, Z. Li, Y. Zhou, C. Fu, and Y. Xiong, "Robust real-time upper limb motion transfer from human to humanoid for manipulation skill teaching," *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 238, no. 5, pp. 1522–1541, 2024.
- [12] M. Alibeigi, S. Rabiee, and M. N. Ahmadabadi, "Inverse kinematics based human mimicking system using skeletal tracking technology," *Journal of Intelligent & Robotic Systems*, vol. 85, no. 1, pp. 27–45, 2017.